動的実行ログを用いたログベース異常検知における自動ログ挿入技術の有効性評価

佐々木 康太[1,a)]　掛井 将平[1,b)]　白石 善明[2]　齋藤 彰一[1]

概要：システムログを用いた異常検知は，正常時と異常時のログ系列差に基づいて異常を判定する．しかし，適切なログ文の実装は容易でない．ログ文を自動生成する自動ログ挿入手法は，主に開発者ログの再現性で評価され，異常検知への有効性は明らかでない．先行研究では，自動挿入ログの有効性を静的解析で生成したログ系列により評価し，正常・異常を切り分ける系列が性能に寄与することを示した．本研究では，実行状態や外部ライブラリの挙動を含む動的実行ログで有効性を検証するため，正常・異常シナリオのログ系列を用い，複数の異常検知手法で効果を評価した．その結果，平均 F1 スコアは DeepLog で 0.000 から最大 0.854，LogRobust で 0.426 から最大 0.859，LogBERT で 0.486 から最大 0.557 となり，いずれも自動挿入ログが開発者ログを上回った．DeepLog では異常判定に必要な系列長を満たす割合が 0.003 から 0.913 以上に増加した．また，ログ挿入位置の除外による性能変化は，先行研究と同様の結果が一貫して得られなかった一方，正常・異常経路を区別する位置のログが異常判定に寄与する事例を確認した．

キーワード：自動ログ挿入，ログベース異常検知

Evaluation of the Effectiveness of Automatic Log Insertion Technology in Log-Based Anomaly Detection Using Dynamic Execution Logs

Kota Sasaki[1,a)]　Shohei Kakei[1,b)]　Yoshiaki Shiraishi[2]　Shoichi Saito[1]

Abstract: Log-based anomaly detection identifies anomalies from differences between normal and anomalous log sequences. However, implementing appropriate logging statements is difficult. Automatic log insertion is mainly evaluated by how well it reproduces developer-written logs, leaving its effectiveness for anomaly detection unclear. Our previous study evaluated automatically inserted logs using statically generated sequences and showed that sequences distinguishing normal from anomalous executions contribute to detection performance. This study evaluates dynamic execution logs reflecting runtime states and external-library behavior, using normal and anomalous scenarios and multiple anomaly detection methods. Mean F1-scores increased from 0.000 to at most 0.854 for DeepLog, 0.426 to 0.859 for LogRobust, and 0.486 to 0.557 for LogBERT. For DeepLog, the proportion of sequences long enough for anomaly detection increased from 0.003 to at least 0.913. Removing logs by insertion location did not consistently reproduce the previous results, while logs distinguishing normal and anomalous paths contributed to anomaly decisions.

Keywords: automatic log insertion, log-based anomaly detection

1. はじめに

バグや脆弱性に起因する意図しない動作や，脆弱性を悪用したサイバー攻撃は継続的な脅威である．近年では，AI の悪用による脆弱性調査や攻撃コード作成の効率化も懸念

[1] 名古屋工業大学, Nagoya Institute of Technology
Gokiso-cho, Showa-ku, Nagoya, Aichi 466-8555, Japan
[2] 神戸大学, Kobe University
1-1 Rokkodai-cho, Nada-ku, Kobe 657-8501, Japan
[a)] k.sasaki.261@nitech.jp
[b)] kakei.shohei@nitech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

されており，IPA は「情報セキュリティ 10 大脅威 2026」[1] に，脆弱性を悪用した攻撃や AI の利用をめぐるサイバーリスクを挙げている．こうした攻撃は，システム運用者に金銭的・社会的損失をもたらすだけでなく，システム利用者の個人情報流出や二次被害につながる可能性がある．そこで，こうした不正動作を早期に検知する手段として，実行時に出力されるログからシステムの状態や振る舞いを分析するログベース異常検知が研究されている．

ログベース異常検知の性能は，システム内部のどの位置でログが出力されるかに依存する．異常に関係する処理が実行されても十分なログがなければ正常実行との差異を観測できない一方，適切な位置へのログ追加によって実行経路や内部状態の違いを観測しやすくなる．しかし，大規模なシステムに対して開発者が適切なログ出力位置を手作業で特定し，ログを追加することは容易ではない．

ソフトウェア工学の分野では，システムにおけるログの役割が異常検知を含め多方面で重要であるとして，プログラム解析に基づいてログ出力処理を自動的に挿入する手法が提案されている．自動ログ挿入により，従来観測できなかった処理の情報をログとして取得でき，ログベース異常検知の性能向上が期待される．我々の先行研究 [2] では，自動ログ挿入の異常検知への有効性を，静的解析に基づいて生成したログ系列を用いて評価した．しかし，静的解析で生成されるログ系列には実際には通過しない経路が含まれる可能性があり，実行時状態やフレームワークの挙動も十分に反映できない．そのため，自動ログ挿入の実際の影響を明らかにするには，実行可能性を反映した動的なログによる評価が必要である．

そこで本研究では，複数の自動ログ挿入手法を適用したアプリケーションに正常・異常シナリオを実行し，得られた動的ログ系列を用いて異常検知性能を評価する．これにより，実際の実行経路や状態を反映したログ系列に基づき，自動ログ挿入の有効性を検証する．

2. 関連研究

2.1 ログベース異常検知

システムが出力するログは，システムの挙動を反映しており，異常動作の証拠となりうるため，ログベース異常検知が研究されている．初期には，ログメッセージの出現回数に対する主成分分析 [3]，ログ間の共起関係から不変条件を抽出する手法 [4]，ログ系列から実行経路をモデル化する手法 [5] などが提案された．その後，ログの出力順序や系列中の文脈を扱うため，深層学習を用いた手法が広く研究されている．

DeepLog [6] は Long Short-Term Memory（LSTM）を用いて過去のログイベント系列から次に出現するイベントを予測，予測結果から逸脱したイベントを異常として検出する．LogRobust [7] は，ログメッセージを自然言語処理によって意味ベクトルへ変換し，Attention 機構を備えた Bi-LSTM によって系列を分類することでログメッセージの変化やノイズに対する頑健性を高めている．LogBERT [8] は，Transformer と Masked Language Modeling を用いて正常なログ系列の文脈を学習し，マスクされたイベントの予測結果に基づいて異常を検出する．

2.2 自動ログ挿入

ソフトウェアシステムの運用においてログは重要である一方，適切なログ文をソースコードへ実装することは容易ではなく，その位置や内容は開発者の経験や方針に依存する [9]．この問題に対し，ログ文の挿入位置 [10]，ログレベル [11]，ログメッセージ [12], [13] といったログ文を構成する要素を自動的に決定する手法が提案されている．

近年では，ログ文を構成する複数の要素を統合的に生成する手法も研究されている．LANCE [14] は，T5（Text-to-Text Transfer Transformer）を用いて，メソッド全体を再生成することで，これらの課題に対処する．一方，メソッド全体の再生成ではログ文以外のコードが変更される可能性や，生成に要する計算コストが課題となる．FastLog [15] は，ソースコードからログ文の挿入位置を予測した後，その位置に必要なログ文のみを生成することで，コード全体を再生成せずにログ文を挿入する．UniLog [16] は汎用 LLM と In-Context Learning（ICL）を組み合わせ，コードの文脈と与えられた例に基づいてログ文を生成する．

3. 研究課題

3.1 研究動機

本研究の目的と先行研究との比較を**図 1** に示す．既存の自動ログ挿入技術は OSS 開発者が実装するログ文をどれだけ再現できるかを評価指標としており，異常検知やデバッグなど，ログ活用時の有効性は十分に評価されていない．我々の先行研究 [2] では，静的解析により生成したログ系列を用いて，自動ログ挿入のログベース異常検知への有効性を評価した．その結果，ログの有効性は分岐などの構文的位置だけでは説明できず，正常経路と異常経路で出現が偏るログ，すなわち実行パス排他性の高いログが異常検知に有効であることが示唆された．一方，正常・異常双方に現れる汎用的なログは誤検知の要因となる場合があった．

しかし，先行研究の評価方法では実行時状態を反映できず，分岐やループにより実際とは異なるログ系列が生成されうる．また，フレームワークや外部ライブラリによるログ・呼び出しを考慮できず，ログ出力位置とメッセージテンプレートが一対一に対応しない場合の関係も評価しにくい．さらに，制御フローグラフに基づいて異常を定義するため，攻撃による不正遷移や不正な決済処理などの意味的な異常を扱いにくい．そこで本研究では，対象アプリケーションを動的実行してログを収集する．

© 2026 Information Processing Society of Japan

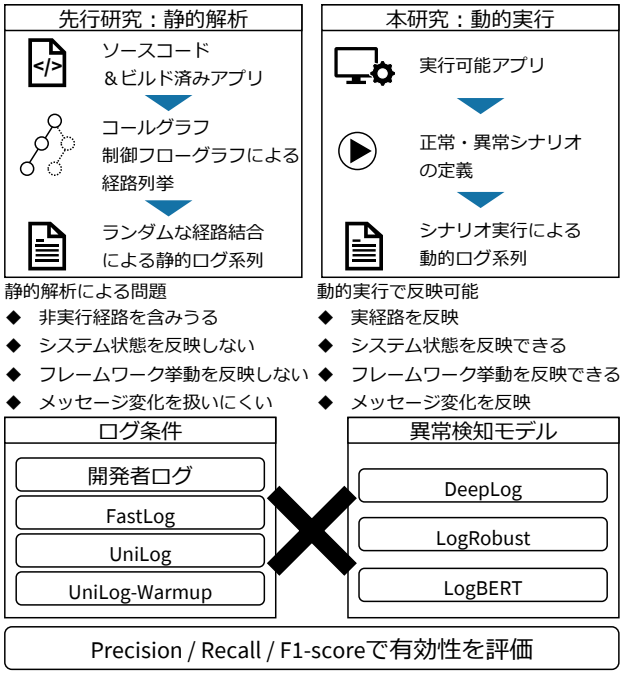


図 1 先行研究との比較と目的
Fig. 1 Comparison with Previous Research and Objectives

3.2 動的実行評価における課題

動的実行ログを用いた評価では，複数のログ条件で同一操作を再現する必要があり，操作コストと再現性が課題となる．また，ログ条件以外の実行条件を共通化しなければ，性能差がログ出力と実行処理のどちらに起因するか区別できない．さらに，観測可能な処理は実行シナリオに依存するため，評価範囲も限定される．

3.3 Research Questions

動的実行ログでは先行研究と同様の傾向が得られるとは限らないため，次の二つのリサーチクエスチョン（RQ）を設定する．

> **RQ1** 動的実行ログにおいて，自動挿入ログは開発者ログよりログベース異常検知性能を向上させるか

RQ1 では，開発者ログと各自動挿入ログの異常検知性能を比較する．自動ログ挿入は正常・異常間の処理経路差を観測しやすくする一方，ログ量や未学習系列の増加により誤検知を増やす可能性がある．複数の自動ログ挿入手法と異常検知手法を用い，動的実行環境での有効性を検証する．

> **RQ2** 静的解析ベース評価で得られたログ挿入位置による傾向は動的実行ログでも再現されるか

RQ2 では，先行研究で確認されたログ挿入位置と異常検知性能の関係を検証する．先行研究の除外型アブレーションを踏まえ，プログラム構造およびソフトウェアレイヤごとにログを除外し，静的解析で得られた傾向が動的実行でも再現されるかを分析する．

4. 動的実行ログデータの収集手法

4.1 全体の流れ

本研究における動的実行ログの収集手順を**図 2** に示す．各自動ログ挿入手法を共通の対象アプリケーションへ適用し，正常・異常シナリオを実行してログを収集する．収集ログに挿入元およびシナリオ実行の識別情報を付与し，評価対象ログの抽出・テンプレート化後，ログ条件ごとに 1 回のシナリオ実行を単位とする系列を構築することで，ログ条件間でアプリケーション実行を共通化する．

4.2 シナリオベース動的実行

4.2.1 シナリオの定義

シナリオは操作主体ごとに階層化した Python ファイルとして実装し，各ファイルに利用目的に対応する操作列とラベルを定義する．各シナリオには正常・異常ラベル，操作主体，操作系列，入力および期待結果を設定する．シナリオランナーは実行中の識別子や状態を保持して後続操作から参照し，実行時状態に依存する操作を実行する．一部のシナリオでは，シナリオ実行 seed に基づき，操作対象，入力値，操作回数および処理を変化させる．各 HTTP 操作には操作 ID を付与し，API カタログの HTTP メソッド・パスとの一致を検証する．

4.2.2 シナリオの実行

各シナリオを setup，measured，verification，cleanup の 4 フェーズに分け，それぞれ事前条件の準備，評価対象操作，結果確認，後処理を行う．異常検知には measured フェーズで出力されたログのみを使用する．

各 HTTP リクエストにはシナリオ ID，実行 ID，フェーズ，操作 ID を付与し，対象アプリケーションで Mapped Diagnostic Context（MDC）を介してログへ伝搬させることで，シナリオ，実行およびフェーズ単位でログを対応付ける．識別情報が付与されない非同期・バックグラウンド処理のログは検証対象外とする．

シナリオランナーは，各操作の応答とアプリケーション状態を期待結果と照合する．異常シナリオでは，HTTP エラーや例外の発生のみでは再現成功とせず，定義した異常状態や期待応答を確認できた場合のみ異常系列とする．また，ログ挿入前に異常の再現を確認し，挿入後にも同一シナリオを実行する．

4.3 ログ収集

アプリケーションログをファイルとして収集し，シナリオ ID および実行 ID に基づき measured フェーズのログのみを抽出する．通常運用を想定し，DEBUG および TRACE レベルは対象外とする．

抽出ログからタイムスタンプ，ログレベル，シナリオ識

© 2026 Information Processing Society of Japan

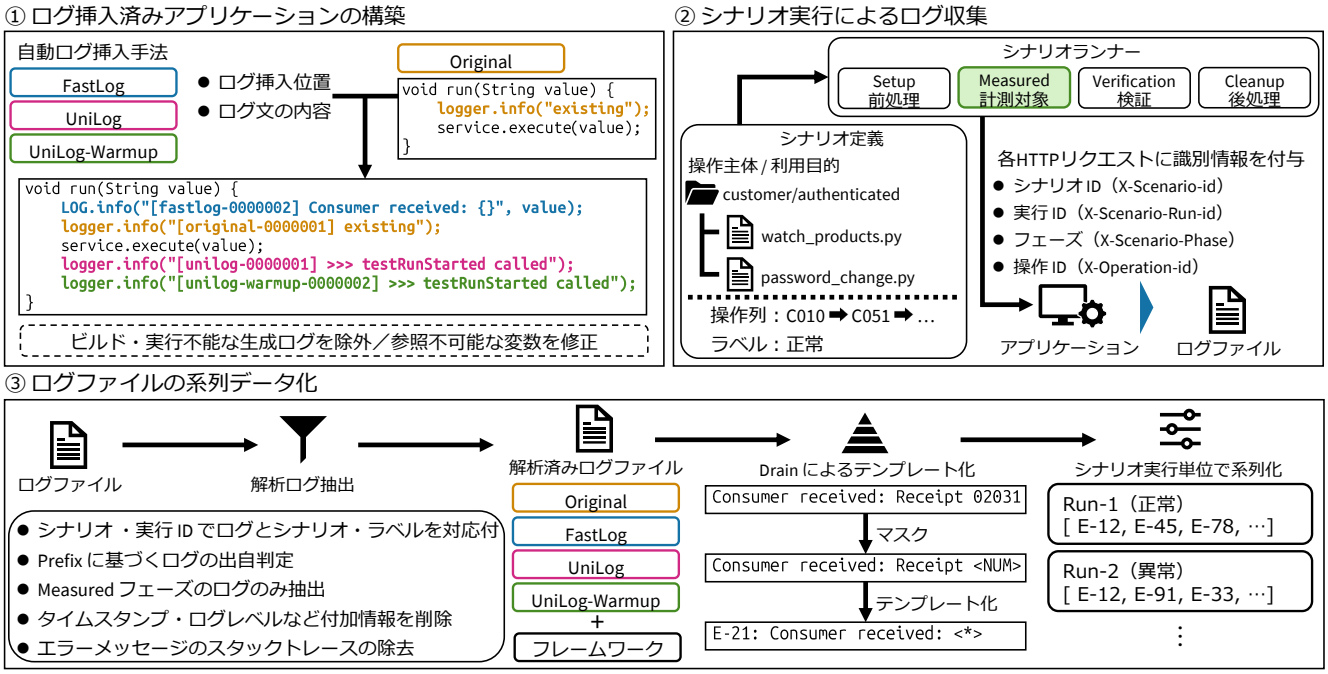


図 2　動的実行ログデータの収集手法の全体像

Fig. 2　Overview of Methods for Collecting Dynamic Execution Log Data

別情報などを分離し，本文のみ抽出する．スタックトレースなどは直前のログに付随する情報として除外する．その後，ログの出自を識別する prefix を削除したうえで，IP アドレス，UUID，メールアドレス，16 進数および数値を抽象化し，収集した評価対象ログ全体に対して Drain [17] を用いてテンプレート化する．Drain の類似度閾値を 0.4，木の深さを 4，各ノードの最大子数を 100 とする．

ログ出力位置は，自動挿入時の情報または開発者ログへの静的解析から対応付ける．同一テンプレートに共通のイベント ID を割り当て，出力順に並べることで，1 回のシナリオ実行を単位とする動的ログ系列を構築する．

4.4　カバレッジの測定

動的実行で観測されるログはシナリオに依存するため，ソースコード上のログ出力点のうち実際に観測された割合を測定する．対象ログ出力点の集合を L_{all}，シナリオ実行時に少なくとも一度観測された集合を L_{exec} とし，ログ出力点カバレッジを $C_{\log} = \frac{||L_{\text{exec}}||}{||L_{\text{all}}||}$ と定義する．L_{all} には，開発者ログでは静的解析で取得した位置，自動挿入ログでは各手法が生成した挿入位置を用い，出力位置を一意に特定できないログは対象外とする．

正常・異常シナリオ群および学習分割で観測されたログ出力点の集合をそれぞれ L_{normal}，L_{anomaly}，L_{train} とする．正常時および異常時に観測されたログ出力点のうち学習分割でも観測された割合を，それぞれ $C_{\text{train}} = \frac{||L_{\text{train}} \cap L_{\text{normal}}||}{||L_{\text{normal}}||}$，$C_{\text{anomaly_seen}} = \frac{||L_{\text{train}} \cap L_{\text{anomaly}}||}{||L_{\text{anomaly}}||}$ と定義する．

5.　実験設定

5.1　対象アプリケーションと実行シナリオ

対象には先行研究と同じオープンソース EC サイト Shopizer [18] を用いる．実験に必要な範囲で一部の実装上の問題を修正し，シナリオ ID や実行フェーズをログへ付加する計測用変更を加えた．自動ログ挿入は修正後のコードに適用し，計測用変更部分は対象外とした．

操作主体と利用目的に基づいて正常・異常シナリオを作成した．異常シナリオは既知の不具合，アクセス制御上の問題，および解析中に確認した異常動作に基づいて定義した．このうち 5 件は経路異常と分類し，対応する正常シナリオと操作主体および操作系列を一致させ，内部経路の差を評価する．一方，ログ系列上の経路差として現れにくいと考えられ，正常シナリオとの一対一の対応を設定していない 9 件を非経路異常とし，経路異常と区別して分析する．なお，これらが実運用環境の異常の種類や発生頻度を代表することは仮定しない．

シナリオとデータ分割を**表 1** に示す．一回のシナリオ実行を 1 セッションとし，シナリオ実行 seed に基づいて学習・検証・評価へ 6 : 2 : 2 で分割する．同一シナリオは各分割に含まれるが，同一シードの実行は重複しない．本研究は未知シナリオへの汎化ではなく，同一ワークロードにおけるログ条件の比較を目的とするため，すべてのログ条件と異常検知モデルで同一分割を用いる．

© 2026 Information Processing Society of Japan

表 1 シナリオおよびデータ分割の構成
Table 1 Scenario and Data Partitioning Settings

種別	シナリオ数	実行回数/件	学習	検証	評価
正常	71	50	2130	710	710
経路異常	5	100	300	100	100
非経路異常	9	50	270	90	90
合計	85	–	2700	900	900

5.2 ログ挿入条件

自動ログ挿入手法には，ログ挿入専用の学習済みモデルを用いる FastLog [15] と，汎用 LLM を用いる UniLog [16] を採用し，開発者ログを削除したソースコードに対して適用した．評価条件は，開発者ログを用いる Original，FastLog，UniLog，追加学習した UniLog を用いる UniLog-Warmup の 4 種類とする．各自動ログ挿入手法が生成したログ自体の有効性を開発者ログと比較するため，各条件では対応するログとフレームワークログのみから系列を構築し，異なる出自のアプリケーションログは混在させない．生成ログと開発者ログには出自識別用 prefix を付与して計測用アプリケーションに統合し，各条件は同一のアプリケーション実行から抽出する．

FastLog には原論文で公開されている学習済みモデルを使用した [15]．UniLog には Llama-3.1-8B-Instruct を用いた．UniLog-Warmup では LANCE [14] データを用いて追加学習した．各手法の詳細な設定を付録 A.2 に示す．

自動生成ログのうち，構文の破壊や到達不能な位置への挿入により対象アプリケーションを実行できないものは除外した．また，参照不可能な変数のみが原因の場合は，当該変数をログ文の引数から除去した．

5.3 異常検知モデル

異なる異常検知方式を採用する DeepLog [6]，LogRobust [7]，LogBERT [8] を用いる．詳細な設定を付録 A.1 に示す．

DeepLog は正常系列から直前 10 イベントに基づく次イベントを学習し，予測上位 9 件から逸脱したイベントを異常とする．LogRobust は FastText [19] と TF-IDF による意味表現で系列を分類し，検証データから異常判定閾値を決定する．LogBERT は正常系列のマスクイベント予測を学習し，異常判定パラメータを検証データから決定する．

各ログ条件で同一のデータ分割とモデル実験 seed を使用し，評価データはハイパーパラメータの決定に使用しない．

5.4 アブレーション条件

RQ2 では，各自動ログ挿入手法の全ログを用いる All 条件を基準に，特定のプログラム構造またはソフトウェアレイヤのログを除外して性能変化を比較する．Branch，Exception，Loop 条件では，それぞれ条件分岐構造内，例外処理周辺，反復処理内のログを除外する．また，Web/API，Adapter，Service，Repository，Integration の各レイヤを除外する条件を設定し，パッケージ構造とログ挿入位置に基づいて分類する．

除外位置そのものの影響とログ出力点数減少の影響を区別するため，各条件で除外されるログ出力点と同数の出力点を無作為に除外する Random 条件を複数の除外 seed で生成して比較する．各条件はアプリケーションを再実行せず，収集済みのログ系列から対象ログを除外して構築する．

表 2 各ログ条件の異常検知性能
Table 2 Anomaly Detection Performance

モデル	ログ条件	Precision	Recall	F1-score
DeepLog	Original	.000 ± .000	.000 ± .000	.000 ± .000
	FastLog	.477 ± .006	.895 ± .000	.622 ± .005
	UniLog	.900 ± .011	.614 ± .061	.729 ± .043
	UniLog-W	.956 ± .004	.772 ± .061	.854 ± .037
LogRobust	Original	.275 ± .003	.947 ± .000	.426 ± .003
	FastLog	.570 ± .084	.667 ± .152	.613 ± .108
	UniLog	.777 ± .078	.965 ± .030	.859 ± .039
	UniLog-W	.335 ± .113	.789 ± .053	.460 ± .096
LogBERT	Original	.652 ± .054	.388 ± .035	.486 ± .042
	FastLog	.787 ± .239	.382 ± .059	.499 ± .051
	UniLog	.762 ± .074	.442 ± .033	.557 ± .013
	UniLog-W	.534 ± .133	.542 ± .070	.525 ± .047

5.5 評価指標

異常検知性能は一回のシナリオ実行を単位とし，正常を Negative，異常を Positive として Precision，Recall，F1 スコアを算出する．

ログが一件も出力されない系列や，モデル固有の判定条件を満たさない系列を単純に除外すると，ログ条件間で評価対象数が異なる．そこで，判定不能系列を正常と判定したものとして全セッションで算出した結果と，判定可能な系列のみを対象とした結果の両方を算出し，後者には判定可能な系列の割合を併記する．

さらに，4.4 節のログ出力点カバレッジを用いて，シナリオ実行および学習データによるログ出力点の被覆状況を分析する．また，DeepLog および LogBERT について，異常判定に用いられたログを記録し，TP および FP における出現をログ単位で集計する．

6. 結果

6.1 RQ1 の回答

各ログ条件における Precision，Recall，F1 スコアを**表 2** に示す．各値は 3 種類のモデル実験 seed における平均値と標準偏差である．

全系列を対象とした平均 F1 スコアは，すべての異常検知モデルで自動ログ挿入条件が Original を上回ったが，改

© 2026 Information Processing Society of Japan

表 3 判定可能系列における F1 スコアと判定可能系列割合
Table 3 F1-score and Scorable Coverage

指標	ログ条件	DeepLog	LogRobust	LogBERT
F1-score	Original	.000 ± .000	.431 ± .003	.596 ± .054
	FastLog	.622 ± .005	.613 ± .108	.499 ± .051
	UniLog	.777 ± .044	.859 ± .039	.560 ± .013
	UniLog-W	.906 ± .037	.460 ± .096	.527 ± .047
Coverage	Original	.003 ± 0.00	.923 ± 0.00	.671 ± .008
	FastLog	.989 ± 0.00	1.000 ± 0.00	.992 ± .002
	UniLog	.949 ± 0.00	1.000 ± 0.00	.996 ± .001
	UniLog-W	.913 ± 0.00	1.000 ± 0.00	.991 ± .004

表 4 異常種別ごとの F1-score（経路異常/非経路異常）
Table 4 F1-score by Anomaly Type

ログ条件	DeepLog	LogRobust	LogBERT
Original	.000/.000	.296/.248	.504/.342
FastLog	.517/.404	.457/.531	.439/.488
UniLog	.866/.451	.791/.737	.554/.463
UniLog-W	.894/.766	.335/.299	.386/.467

善幅はログ挿入手法とモデルの組合せによって異なった．特に，静的解析ベースの先行研究では自動挿入により性能が低下した LogRobust でも，本研究ではすべての自動挿入条件で平均 F1 スコアが Original を上回った．先行研究では到達可能な経路を組み合わせて静的系列を生成するため，実際の一連の実行では成立しないログ間の共起によって，系列全体の意味的な整合性が損なわれた可能性がある．意味表現を用いる LogRobust では，実際の処理に沿った動的系列を用いることでこの影響が軽減された可能性がある．ただし，先行研究とは異常定義やモデル設定等も異なるため，経路結合のみを原因とすることはできない．

判定可能系列のみを対象とした F1 スコアとその割合を**表 3** に示す．DeepLog および LogBERT では，自動ログ挿入によって判定可能系列が大きく増加した．特に DeepLog では，Original の多くの系列が次イベント予測に必要な長さを満たしておらず，自動ログ挿入には系列モデルを適用可能な観測粒度を確保する効果もあった．一方，判定可能系列のみでは LogBERT の Original がすべての自動挿入条件を上回るため，LogBERT の全系列での改善は主に判定可能系列の増加によると考えられる．LogRobust の UniLog では判定可能系列のみでも改善しており，識別性能自体も向上した．

さらに，同一セッションの判定変化を確認したところ，DeepLog では主として新たな異常系列を検知可能になり，LogRobust の UniLog では主として FP の減少が性能改善につながった．LogBERT では新たな TP の獲得と既存 TP の消失がともに確認された．

各異常種別を正例，全正常評価系列を負例として算出した F1 スコアの 3 種類のモデル実験 seed における平均を**表 4** に示す．経路異常・非経路異常の双方で，Original を上回る自動挿入条件が確認された．ただし，非経路異常では正常シナリオと操作主体・操作系列を一致させた比較を行っていないため，内部経路差に対する自動ログ挿入の効果は経路異常の結果がより直接的な評価となる．

以上より RQ1 に対して，自動挿入ログを用いることで動的実行ログにおける異常検知性能を改善し得るが，その効果は挿入手法とモデルに依存する．また，性能改善には識別性能の変化だけでなく，判定可能な系列を増加させる効果も含まれる．

6.2 RQ2 の回答

6.2.1 アブレーション条件による分析

各アブレーション条件における $\Delta F1 = F1_{\text{ablation}} - F1_{\text{All}}$ の平均値と標準偏差を**表 5** に示す．Random $\Delta F1$ は，各条件と同数のログ出力点を無作為に除外し，3 種類ずつの除外 seed とモデル実験 seed の全 9 組合せから算出した．

Branch ログの除外では，FastLog の DeepLog および LogRobust，UniLog の DeepLog および LogBERT などで，Random 除外より平均 F1 スコアが大きく低下した．一方，Exception，Loop および各ソフトウェアレイヤではモデルとログ条件によって傾向が異なり，特定カテゴリのログが一様に有効とはいえなかった．

この結果は先行研究とは一部異なる．先行研究ではソースコード上の例外生成・送出などを基準に異常系列を定義したため，Exception が異常と結び付きやすかった．本研究では期待結果から異常を定義するため，正常に回復する例外処理も正常に含まれ，Exception ログが先行研究ほど異常経路に特異的でなかった可能性がある．

逆に Branch では，静的解析で到達可能な分岐を組み合わせる先行研究に対し，本研究では入力と状態によって選択された分岐のみが現れる．このため，正常・異常で経路が分かれる Branch ログが明確な観測点となった可能性がある．すなわち，ログの有効性は Branch や Exception という構文カテゴリ自体ではなく，異常の定義と実際の実行経路との関係によって変化すると考えられる．

6.2.2 ログ単位の分析

位置カテゴリだけでは性能変化を説明できないため，DeepLog および LogBERT について，TP・FP 判定に関与したログの出力位置を確認した．以下では，TP または FP への関与に偏りが見られたログを代表例として示す．

FastLog では，対象が存在しない場合にのみ通過する分岐内のあるログが，DeepLog で 120 回，LogBERT で 32 回 TP 判定に関与し，FP では確認されなかった．一方，正常実行でも通過する処理に挿入された別のログは，DeepLog で TP には関与せず，FP に 60 回関与した．

UniLog では，異常実行で選択される経路上のログが，DeepLog で 260 回，LogBERT で 112 回 TP 判定に関与し，FP では確認されなかった．同一テンプレートのログは他

© 2026 Information Processing Society of Japan

表 5 ログ除外による F1-score の変化
Table 5 Changes in F1-score by Log Removal

		DeepLog		LogRobust		LogBERT	
ログ条件	除外対象	ΔF1-score	Random ΔF1	ΔF1-score	Random ΔF1	ΔF1-score	Random ΔF1
FastLog	Branch	−.105 ± .060	+.034 ± .078	−.115 ± .088	+.041 ± .133	−.005 ± .054	+.004 ± .050
	Exception	−.028 ± .049	−.050 ± .088	−.058 ± .175	−.059 ± .105	−.044 ± .050	+.005 ± .054
	Loop	+.009 ± .032	−.050 ± .088	+.012 ± .040	−.059 ± .105	+.012 ± .059	+.005 ± .054
	Web/API	+.022 ± .025	+.004 ± .010	−.084 ± .138	−.017 ± .069	+.071 ± .042	+.013 ± .034
	Adapter	+.007 ± .016	+.060 ± .086	−.003 ± .046	−.012 ± .122	+.010 ± .023	+.019 ± .064
	Service	+.009 ± .026	−.028 ± .091	−.051 ± .039	−.048 ± .080	+.038 ± .074	+.010 ± .077
	Repository	+.004 ± .013	+.008 ± .012	+.036 ± .023	−.001 ± .023	+.022 ± .066	+.007 ± .019
	Integration	+.015 ± .005	−.079 ± .061	−.055 ± .147	+.013 ± .105	+.022 ± .037	+.009 ± .065
UniLog	Branch	−.148 ± .053	+.062 ± .040	−.011 ± .063	−.070 ± .118	−.058 ± .030	+.011 ± .049
	Exception	+.035 ± .086	+.063 ± .042	−.045 ± .023	−.014 ± .047	+.021 ± .051	+.003 ± .043
	Loop	+.030 ± .042	+.063 ± .042	+.020 ± .020	−.014 ± .047	+.009 ± .016	.000 ± .066
	Web/API	+.003 ± .006	+.013 ± .046	−.027 ± .021	−.046 ± .105	+.024 ± .044	+.012 ± .022
	Adapter	+.076 ± .048	+.036 ± .034	+.001 ± .025	−.026 ± .044	+.013 ± .038	+.007 ± .056
	Service	+.036 ± .087	−.036 ± .127	+.007 ± .047	−.019 ± .048	+.021 ± .039	−.060 ± .049
	Repository	+.007 ± .075	+.020 ± .051	−.028 ± .022	−.009 ± .022	−.029 ± .024	−.002 ± .022
	Integration	−.034 ± .007	+.050 ± .082	−.009 ± .011	−.024 ± .140	+.006 ± .051	−.019 ± .054
UniLog-Warmup	Branch	−.010 ± .081	−.063 ± .035	+.043 ± .095	−.032 ± .147	+.002 ± .024	−.018 ± .055
	Exception	+.035 ± .037	−.206 ± .131	−.029 ± .019	+.014 ± .073	−.001 ± .057	−.069 ± .068
	Loop	+.015 ± .066	−.206 ± .131	+.114 ± .024	+.014 ± .073	+.020 ± .064	−.069 ± .068
	Web/API	−.020 ± .054	+.001 ± .002	−.011 ± .120	+.012 ± .029	−.008 ± .042	+.002 ± .035
	Adapter	+.037 ± .076	+.022 ± .045	+.032 ± .049	+.013 ± .054	+.006 ± .043	+.006 ± .037
	Service	+.013 ± .117	−.084 ± .173	+.019 ± .029	+.029 ± .096	−.010 ± .017	−.041 ± .095
	Repository	−.022 ± .078	−.002 ± .063	+.002 ± .015	+.016 ± .059	−.007 ± .025	+.015 ± .035
	Integration	−.005 ± .073	−.117 ± .172	+.044 ± .099	+.008 ± .085	−.007 ± .014	−.045 ± .068

表 6 ログ出力点カバレッジ
Table 6 Log Output Point Coverage

ログ条件	C_{log}	C_{train}	$C_{anomaly_seen}$
Original	23/ 398	20/ 20	7/ 10
FastLog	312/6,183	300/300	93/105
UniLog	303/2,907	296/296	76/ 83
UniLog-Warmup	295/2,379	288/288	71/ 78

の複数位置にも生成されていたが，それらでは同様の TP への関与は確認されなかった．また，正常実行で観測される別のログは，LogBERT で FP 判定にのみ関与した．

以上から，自動挿入ログの異常判定への関与は出力位置によって異なり，同一テンプレートでも位置によって TP への関与が異なった．これらの事例は，ログの有効性に正常・異常経路上での出現位置が重要であることを示唆しており，先行研究の実行パス排他性に関する知見と整合する．

以上より RQ2 に対して，静的解析ベース評価で得られたログ挿入位置の傾向は動的実行では一様に再現されなかった．一方，正常・異常経路を区別できる位置のログが重要という先行研究の知見とは整合した．

6.3 カバレッジに基づく補助分析

ログ出力点カバレッジの測定結果を**表 6** に示す．

動的実行で観測されたログ出力点は全条件で限定的であり，評価結果は実行シナリオの範囲に依存する．一方，正常シナリオで観測された出力点は全条件で学習分割にも含まれ，自動挿入条件では異常時の出力点の大部分も学習分割で観測された．したがって，性能差は学習分割で未観測の出力点の出現だけでなく，実行経路上のログの順序や組合せにも依存した可能性がある．なお，本指標は分割内での出力点の被覆を表すものであり，各モデルが当該ログを学習したことや，未知シナリオへの汎化性能を意味しない．

6.4 本研究の制限と今後の課題

本研究は Shopizer のみを対象とし，実験に必要な不具合修正や実行を妨げる自動生成ログの除外・修正も行っているため，他のアプリケーションで同様の結果が得られるとは限らない．

動的実行で観測できる処理はシナリオに依存し，全機能・状態遷移や未知シナリオへの汎化，非同期処理を網羅していない．異常シナリオも実環境を代表するものではなく，経路異常は 5 件に限られ，攻撃シナリオも体系的に扱っていない．今後はカバレッジに基づくシナリオ拡充と，複数アプリケーション・攻撃異常での評価が必要である．

先行研究とは系列生成方法のほか，異常定義，データ分

© 2026 Information Processing Society of Japan

割，アブレーション対象，モデル設定等も異なり，静的・動的評価の差を系列生成方法だけに帰属できない．さらに，異常検知手法とモデル実験 seed 数は限定され，Random との差について統計的有意性も検証していない．

7. まとめ

本研究では，動的実行ログを用いて自動ログ挿入のログベース異常検知への有効性を評価した．Shopizer を対象とした評価では，全系列での平均 F1 スコアはすべての自動挿入条件で Original を上回ったが，効果はログ挿入手法と異常検知モデルによって異なった．また，静的解析で得られた位置カテゴリの傾向は動的実行では一様に再現されなかった一方，ログ単位の分析は，正常・異常経路を区別できるログが重要という先行研究の知見と整合した．今後は対象アプリケーションとシナリオを拡張し，攻撃異常を含む多様な異常で評価する．

謝辞 本研究は JSPS 科研費 JP25K21199 による研究成果を一部含む．

参考文献

[1] 独立行政法人情報処理推進機構：情報セキュリティ 10 大脅威 2026 解説書［組織編］，`https://www.ipa.go.jp/security/10threats/omgdg50000008fi8-att/kaisetsu_2026_soshiki.pdf` (2026). Accessed: 2026-08-15.

[2] 佐々木康太，掛井将平，齋藤彰一：ログシミュレータを用いた自動ログ挿入技術の異常検知タスクへの適用可能性の予備的評価，信学技報 IA2026-19, ICSS2026-19，電子情報通信学会 (2026).

[3] Xu, W., Huang, L., Fox, A. et al.: Detecting large-scale system problems by mining console logs, *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, SOSP '09, New York, NY, USA, Association for Computing Machinery, pp. 117–132 (online), DOI: 10.1145/1629575.1629587 (2009).

[4] Lou, J.-G., Fu, Q., Yang, S. et al.: Mining Invariants from Console Logs for System Problem Detection, *2010 USENIX Annual Technical Conference (USENIX ATC 10)*, USENIX Association, pp. 1–14 (online), available from ⟨https://www.usenix.org/conference/usenix-atc-10/mining-invariants-console-logs-system-problem-detection⟩ (2010).

[5] Yu, X., Joshi, P., Xu, J. et al.: CloudSeer: Workflow Monitoring of Cloud Infrastructures via Interleaved Logs, *SIGARCH Comput. Archit. News*, Vol. 44, No. 2, pp. 489–502 (online), DOI: 10.1145/2980024.2872407 (2016).

[6] Du, M., Li, F., Zheng, G. et al.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, New York, NY, USA, Association for Computing Machinery, pp. 1285–1298 (online), DOI: 10.1145/3133956.3134015 (2017).

[7] Zhang, X., Xu, Y., Lin, Q. et al.: Robust log-based anomaly detection on unstable log data, *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2019, New York, NY, USA, Association for Computing Machinery, pp. 807–817 (online), DOI: 10.1145/3338906.3338931 (2019).

[8] Guo, H., Yuan, S. and Wu, X.: LogBERT: Log Anomaly Detection via BERT, *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8 (online), DOI: 10.1109/IJCNN52387.2021.9534113 (2021).

[9] Yuan, D., Park, S. and Zhou, Y.: Characterizing logging practices in open-source software, *2012 34th International Conference on Software Engineering (ICSE)*, pp. 102–112 (online), DOI: 10.1109/ICSE.2012.6227202 (2012).

[10] Li, Z., Chen, T.-H. P. and Shang, W.: Where shall we log? studying and suggesting logging locations in code blocks, *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ASE '20, New York, NY, USA, Association for Computing Machinery, pp. 361–372 (online), DOI: 10.1145/3324884.3416636 (2021).

[11] Li, Z., Li, H., Chen, T.-H. et al.: DeepLV: Suggesting Log Levels Using Ordinal Based Neural Networks, *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 1461–1472 (online), DOI: 10.1109/ICSE43902.2021.00131 (2021).

[12] Liu, Z., Xia, X., Lo, D. et al.: Which Variables Should I Log?, *IEEE Transactions on Software Engineering*, Vol. 47, No. 9, pp. 2012–2031 (online), DOI: 10.1109/TSE.2019.2941943 (2021).

[13] Ding, Z., Li, H. and Shang, W.: LoGenText: Automatically Generating Logging Texts Using Neural Machine Translation, *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 349–360 (online), DOI: 10.1109/SANER53432.2022.00051 (2022).

[14] Mastropaolo, A., Pascarella, L. and Bavota, G.: Using deep learning to generate complete log statements, *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, New York, NY, USA, Association for Computing Machinery, pp. 2279–2290 (online), DOI: 10.1145/3510003.3511561 (2022).

[15] Xie, X., Cai, Z., Chen, S. et al.: FastLog: An End-to-End Method to Efficiently Generate and Insert Logging Statements, *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, New York, NY, USA, Association for Computing Machinery, pp. 26–37 (online), DOI: 10.1145/3650212.3652107 (2024).

[16] Xu, J., Cui, Z., Zhao, Y. et al.: UniLog: Automatic Logging via LLM and In-Context Learning, *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, Association for Computing Machinery, pp. 1–12 (online), DOI: 10.1145/3597503.3623326 (2024).

[17] He, P., Zhu, J., Zheng, Z. et al.: Drain: An Online Log Parsing Approach with Fixed Depth Tree, *2017 IEEE International Conference on Web Services (ICWS)*, pp. 33–40 (online), DOI: 10.1109/ICWS.2017.13 (2017).

[18] shopizer-ecommerce: Shopizer (2025). v3.2.7, available from `https://github.com/shopizer-ecommerce/shopizer` (accessed 2026-05-12).

[19] Grave, E., Bojanowski, P., Gupta, P. et al.: Learning Word Vectors for 157 Languages, *Proceedings of the International Conference on Language Resources and Evaluation (LREC 2018)*, pp. 3483–3487 (2018).

© 2026 Information Processing Society of Japan

付　　録

A.1　異常検知モデルの詳細設定

本付録では，本文 5.3 節で用いた DeepLog，LogRobust，LogBERT の実装および学習・推論設定を示す．各モデルでは同一のデータ分割を用い，評価データはモデル選択や異常判定パラメータの決定に使用しない．

A.1.1　共通設定

各実験では一つのモデル実験 seed から，モデル初期化用 seed，データ順序用 seed，LogBERT のマスク生成用 seed を SHA-256 に基づいて決定論的に導出する．モデル初期化前に Python，NumPy，PyTorch へ seed を設定し，CUDA 利用時は各 GPU にも seed を設定する．また，PyTorch の cuDNN では決定論的実行を有効化し，benchmark を無効化する．

各モデルは検証損失が 10 エポック連続で改善しない場合に early stopping を行い，検証損失が最小となった checkpoint を評価に用いる．ログ条件間では同一のデータ分割およびモデル実験 seed を使用する．

Event ID の語彙は，各ログ条件・アブレーション条件について学習・検証・評価分割で観測された Event ID の和集合から構築する．本研究では未知 Event ID の処理性能ではなくログ条件間の比較を目的とし，学習分割に出現しないイベントも個別の Event ID として区別したまま評価するため，全分割で観測された Event ID の集合から語彙を固定する．語彙構築には Event ID の集合のみを用い，評価ラベル，出現頻度および系列順序は使用しない．共通語彙では`<PAD>`を token ID 0 として予約する．

A.1.2　DeepLog

DeepLog では，Event ID を one-hot 表現へ変換し，直前 10 イベントを入力として次イベントを予測する．`<PAD>`に対応する one-hot 列は除去されるため，padding 位置は全要素 0 のベクトルとして入力される．モデルは 2 層 LSTM と線形分類層から構成する．

学習では正常系列から長さ 10 の履歴と次イベントの組を生成し，Cross Entropy を最小化する．推論時には，各予測位置で実イベントが予測確率上位 9 件に含まれない場合を異常位置とし，一つ以上の異常位置を含むセッションを異常と判定する．空系列は `empty_sequence`，系列長が 10 以下の系列は `too_short` として判定不能とする．

A.1.3　LogRobust

LogRobust では，各 Event ID に対応するログテンプレートから FastText [19] と TF-IDF を用いて 300 次元の意味ベクトルを事前に生成する．IDF は学習分割のみから算出し，同一の IDF を学習・検証・評価分割へ適用する．生成済み意味ベクトルが存在しない Event ID を含む系列は判定不能とする．

モデルは 1 層 Bi-LSTM と Attention，2 クラス分類層から構成する．時刻 t の Bi-LSTM 出力を $\boldsymbol{h}_t$ とすると，Attention は

$$\alpha_t = \tanh(\boldsymbol{W}_\alpha \boldsymbol{h}_t) \tag{A.1}$$

とし，softmax による時刻方向の正規化は行わない．系列表現は

$$\boldsymbol{v} = \sum_t \alpha_t \boldsymbol{h}_t \tag{A.2}$$

として計算する．

学習では，判定可能な異常系列をすべて使用し，同数の判定可能な正常系列をモデル実験 seed に基づいて抽出する．判定可能な正常系列が不足する場合は balanced sampling による学習を行わない．また，学習分割および検証分割の双方で正常・異常の両クラスが判定可能であることを要求する．

推論では softmax 後の異常クラス確率を異常スコアとする．異常判定閾値は，判定可能な検証系列のみを対象として F1 スコアを最大化するように選択する．候補値は，検証系列で得られた異常確率に 0，0.5，1 を加えた集合とする．F1 スコアが同一の場合は 0.5 に近い閾値を優先し，さらに同率の場合は小さい閾値を選択する．空系列は `empty_sequence`，意味ベクトルを利用できない Event ID を含む系列は `vector_unavailable` として判定不能とする．

A.1.4　LogBERT

LogBERT では，Event ID 系列に対して Masked Language Modeling に基づくイベント予測と，正常系列表現を中心付近へ集約する学習を行う．LogBERT 用語彙で

表 A·1　DeepLog の詳細設定
Table A·1　Detailed Settings of DeepLog

設定	値
入力表現	one-hot Event ID
履歴長	10
LSTM 層数	2
Hidden size	64
Dropout	0.0
損失関数	Cross Entropy
学習データ	正常系列のみ
Optimizer	Adam
Learning rate	0.001
Batch size	512
最大 epoch	100
Early stopping patience	10
異常判定	Top-9

© 2026 Information Processing Society of Japan

表 A·2 LogRobust の詳細設定
Table A·2 Detailed Settings of LogRobust

設定	値
入力表現	FastText + TF-IDF
意味ベクトル次元	300
Bi-LSTM 層数	1
Hidden size	64
Bidirectional	Yes
Dropout	0.0
Attention	非 softmax 型
出力クラス数	2
損失関数	Cross Entropy
学習データ	正常＋異常
Sampling	Balanced
Optimizer	SGD
Learning rate	0.01
Momentum	0.9
Weight decay	1.0×10^{-4}
Gradient clipping	max norm 1.0
Batch size	128
最大 epoch	50
Early stopping patience	10

表 A·3 LogBERT の詳細設定
Table A·3 Detailed Settings of LogBERT

設定	値
Input embedding dimension	50
Hidden dimension	256
Transformer layers	2
Attention heads	4
FFN dimension	512
Dropout	0.1
最大系列長	512
最大イベント数/chunk	511
学習データ	正常系列のみ
Mask ratio m	0.3
VHM weight α	0.01
Optimizer	Adam
Learning rate	0.001
Adam (β_1, β_2)	$(0.9, 0.999)$
Weight decay	0.0
Scheduler	None
Warmup steps	0
Batch size	64 sessions
Forward chunk batch size	64 chunks
最大 epoch	100
Early stopping patience	10

は<PAD>，<MASK>，<DIST>をそれぞれ token ID 0，1，2 として予約し，Event ID には 3 以降を割り当てる．

最大系列長は 512 とし，各 chunk の先頭に<DIST>を追加するため，一つの chunk に含めるログイベント数は最大 511 とする．511 イベントを超えるセッションは複数 chunk へ分割し，セッション表現には各 chunk の<DIST>位置の表現の平均を用いる．

入力 token は 50 次元の embedding と sinusoidal positional encoding を加算した後，線形層によって 256 次元へ射影して Transformer へ入力する．学習時には各イベントを $m = 0.3$ の割合で決定論的にマスクし，マスクされた Event ID に対する平均 Cross Entropy を Masked Log Key Prediction（MLKP）損失とする．マスク位置はマスク用 seed，セッション・chunk の識別情報，epoch から決定論的に生成する．

正常系列表現の中心 $\boldsymbol{c}$ は，マスクを行わない正常学習系列のみから算出する．各セッションの表現を $\boldsymbol{z}_i$ とし，中心からの二乗 Euclidean 距離の平均を

$$L_{\mathrm{VHM}} = \frac{1}{N} \sum_{i=1}^{N} \|\boldsymbol{z}_i - \boldsymbol{c}\|_2^2 \qquad \text{(A.3)}$$

とする．総損失は

$$L = L_{\mathrm{MLKP}} + \alpha L_{\mathrm{VHM}}, \qquad \alpha = 0.01 \qquad \text{(A.4)}$$

とする．

m および α は**表 A·3** の固定値を使用する．異常判定パラメータ g, r は，判定可能な検証系列のみを対象として F1 スコアが最大となる組を選択する．Event ID 数を超える g は候補から除外し，F1 スコアが同一の場合は小さい g，次いで小さい r を優先する．

表 A·4 LogBERT の異常判定パラメータ候補
Table A·4 Candidate Parameters for LogBERT Detection

パラメータ	候補値
g	$\{1, 3, 5, 7, 9, 11, 13, 15, 17, 20\}$
r	$\{0, 1, 3, 5, 10, 20, 50, 100, 200, 500\}$

推論時にも $m = 0.3$ で決定論的にマスク位置を選択する．各マスク位置について，実 Event ID の予測順位が g を超える場合を異常イベントとし，セッション内の異常イベント数が r を超えた場合に異常セッションと判定する．空系列は `empty_sequence`，イベントを含むもののマスク対象が一つも選択されなかった系列は `no_masked_events` として判定不能とする．VHM による中心からの距離は記録するが，最終的な異常判定はマスクイベントの予測順位と g, r に基づいて行う．

A.2 自動ログ挿入手法の詳細設定

本付録では，本文 5.2 節で用いた FastLog，UniLog，UniLog-Warmup の生成設定を示す．いずれの条件も開発者ログを削除した同一の対象ソースコードに適用し，生成されたログ文のみを各自動挿入条件のログとして使用する．

© 2026 Information Processing Society of Japan

表 A·5 FastLog の設定
Table A·5 Settings of FastLog

設定	値
モデル	公開済み学習モデル
追加学習	なし
入力	開発者ログを削除したソースコード
出力	ログ挿入位置およびログ文

表 A·6 UniLog の設定
Table A·6 Settings of UniLog

設定	値
基盤モデル	Llama-3.1-8B-Instruct
生成方法	In-Context Learning
ICL データ	LANCE 学習データ
類似度算出	CodeBERT
ICL 例数	5
例の選択	類似度上位 5 件
入力	開発者ログを削除したソースコード

表 A·7 UniLog-Warmup の追加学習設定
Table A·7 Fine-tuning Settings of UniLog-Warmup

設定	値
基盤モデル	Llama-3.1-8B-Instruct
追加学習方式	LoRA
追加学習データ	LANCE 検証データ
データ数	500
Epoch	3
Learning rate	2×10^{-5}
Effective batch size	8
量子化	4bit

A.2.1 FastLog

FastLog [15] には，原論文で公開されている学習済みモデルを使用する．追加学習は行わず，公開モデルを用いてログ挿入位置および各位置に挿入するログ文を生成する．

A.2.2 UniLog

UniLog [16] では，基盤モデルとして Llama-3.1-8B-Instruct を使用する．ログ生成時には In-Context Learning（ICL）を用い，対象コードと LANCE [14] の学習データとの類似度を CodeBERT によって算出する．類似度の高い 5 件を ICL の例として与え，対象コードに対するログ文を生成する．

A.2.3 UniLog-Warmup

UniLog-Warmup では，UniLog と同じ Llama-3.1-8B-Instruct を基盤モデルとし，ログ生成前に LANCE の検証データ 500 件を用いて Low-Rank Adaptation（LoRA）による追加学習を行う．追加学習は 3 エポック，学習率 2×10^{-5}，実効バッチサイズ 8 とし，4bit 量子化を用いる．追加学習後のモデルを用いて UniLog と同様にログを生成する．

© 2026 Information Processing Society of Japan